

DATABASE PROJECT - PROF. VINCENZO MOSCATO



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

PASS-PAR-TOUT: SMART MOBILITY



Edited by:

de Pertis Giorgio N46005592

de Pasquale Marco N46005559

Capponcelli Nicole Yuki N46006700

TABLE OF CONTENTS

1 INTRODUCTION

- 1.1 Who we are
- 1.2 How it works

2 SPECIFICATIONS

- 2.1 Data specifications
- 2.2 Operation specifications

3 DATABASE DESIGN

- 3.1 Specification analysis and requirements restructuring
- 3.2 Conceptual design
- 3.3 Logical design
 - 3.3.1 Transformation phase
 - 3.3.2 Translation phase
- 3.4 Physical design (create table)

4 APPLICATION DESIGN

- 4.1 Data tier
- 4.2 Application tier
- 4.3 Presentation tier

5 OPTIMISATION

- 5.1 Choosing the optimal concurrency control method
- 5.2 Strategies to ensure reliability

6 SQL

- 6.1 Stored procedures
- 6.2 Queries
- 6.3 Views
- 6.4 Triggers

1 INTRODUCTION

1.1 WHO ARE WE?

In a world ruled by stress and tight schedules, our goal is to make your car journeys better, freeing them from potential causes of delay due to endless traffic queues. Today, information systems cover most of our daily lives: it has therefore become necessary to keep pace with the technological progress of our era and to apply these advances to the optimisation of motorway electronic toll collection, which can accompany us throughout the entire working week. Considering also that electronic payments are spreading more and more among users, we have packaged all these factors into a single “pass-par-tout” device.

1.2 HOW DOES IT WORK?

The DEVICE we propose can be associated with a maximum of two cars belonging to a given CUSTOMER. In full compliance with privacy regulations, the personal details of the customer and the related billing information for payment will be recorded. Every customer who chooses our electronic toll payment method must also provide the licence plate and the model of their CAR(S).

Billing is based on the JOURNEY made, which is identified by a number. The entry TOLL BOOTH onto the motorway, with its date and time, and the exit TOLL BOOTH, with its date and time, are then recorded.

2 SPECIFICATIONS

2.1 DATA SPECIFICATIONS

DEVICES must have a unique code for their identification, and each one can be associated with a maximum of two cars belonging to a given CUSTOMER. The customer must be registered with their personal details and billing information, and can choose either a credit card or a bank account as the payment method.

For each device, the JOURNEY made must also be stored in order to bill the customer. A journey is identified by the details of the entry toll booth, with its date and time, and of the exit toll booth, with its date and time.

A TOLL BOOTH is identified by a motorway exit code and a number. Finally, a CAR is stored with its licence plate, its model and its owner, in case the owner is one of the customers.

2.2 OPERATION SPECIFICATIONS

The main operations planned on our database are:

- insertion of new customers
- insertion of their personal data
- insertion/update of car data
- insertion/update of payment data

- update of the journey made
- update of the toll booths used
- update of the date and time of the toll booths used
- insertion of toll booth data
- viewing of their own journeys by the customer
- viewing of registered customers by the company

3 DATABASE DESIGN

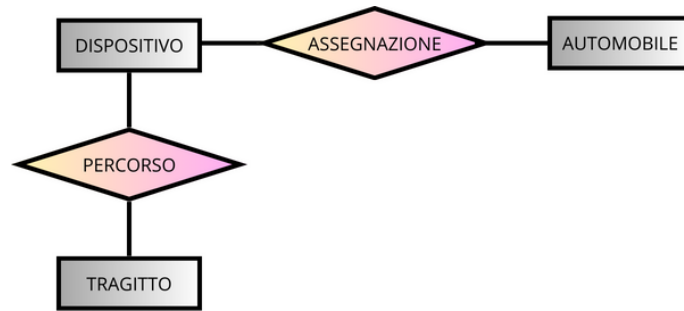
3.1 SPECIFICATION ANALYSIS AND REQUIREMENTS RESTRUCTURING

Before moving on to conceptual design, it is important to have a clear view of the information to be managed:

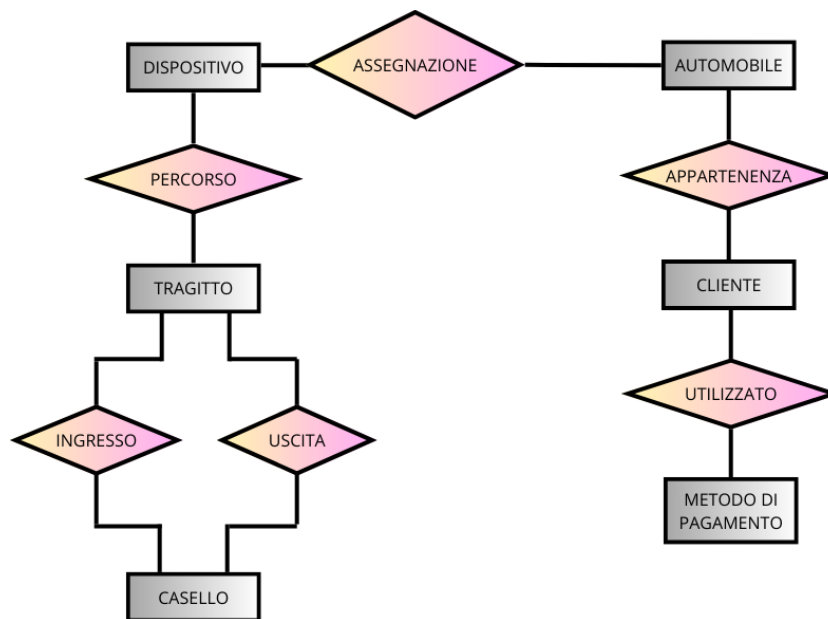
- **Customer information:** the personal details (id_cliente, nome, cognome, data_di_nascita and indirizzo — i.e. customer ID, first name, surname, date of birth and address) are stored for each customer.
- **Car information:** namely the licence plate, the model and the owner, in case the owner is a customer.
- **Device information:** the device is uniquely identified by a codice_dispositivo (device code).
- **Journey information:** it is uniquely identified by a numero_percorso (route number) and by the codice_dispositivo of the device that made that journey.
- **Toll booth information:** toll booths are associated with the journey depending on whether, for that given device, they are entry or exit booths. In both cases, the passage through the booth is characterised by a date and time. Each toll booth also has a numero_casello (booth number) and a nome_uscita (exit name).
- **Payment method information:** it has an id_metpag assigned by the DBA and can be either a credit card (with a numero_carta — card number — and an expiry date) or a current account (with a numero_conto — account number).

3.2 CONCEPTUAL DESIGN

For the conceptual design a mixed strategy was used, characterised by an initial TOP DOWN approach aimed at identifying the fundamental concepts and building a skeleton schema, followed by a refinement process carried out with a typically BOTTOM UP approach. Specifically, a first analysis led to the definition of the entities DISPOSITIVO (device), AUTOMOBILE (car) and TRAGITTO (journey), linked to each other through the relationships ASSEGNAZIONE (assignment) and PERCORSO (route):



Going deeper into the specifications, the need emerged to also consider further entities, namely CASELLO (toll booth), CLIENTE (customer) and METODO DI PAGAMENTO (payment method), linked as follows:



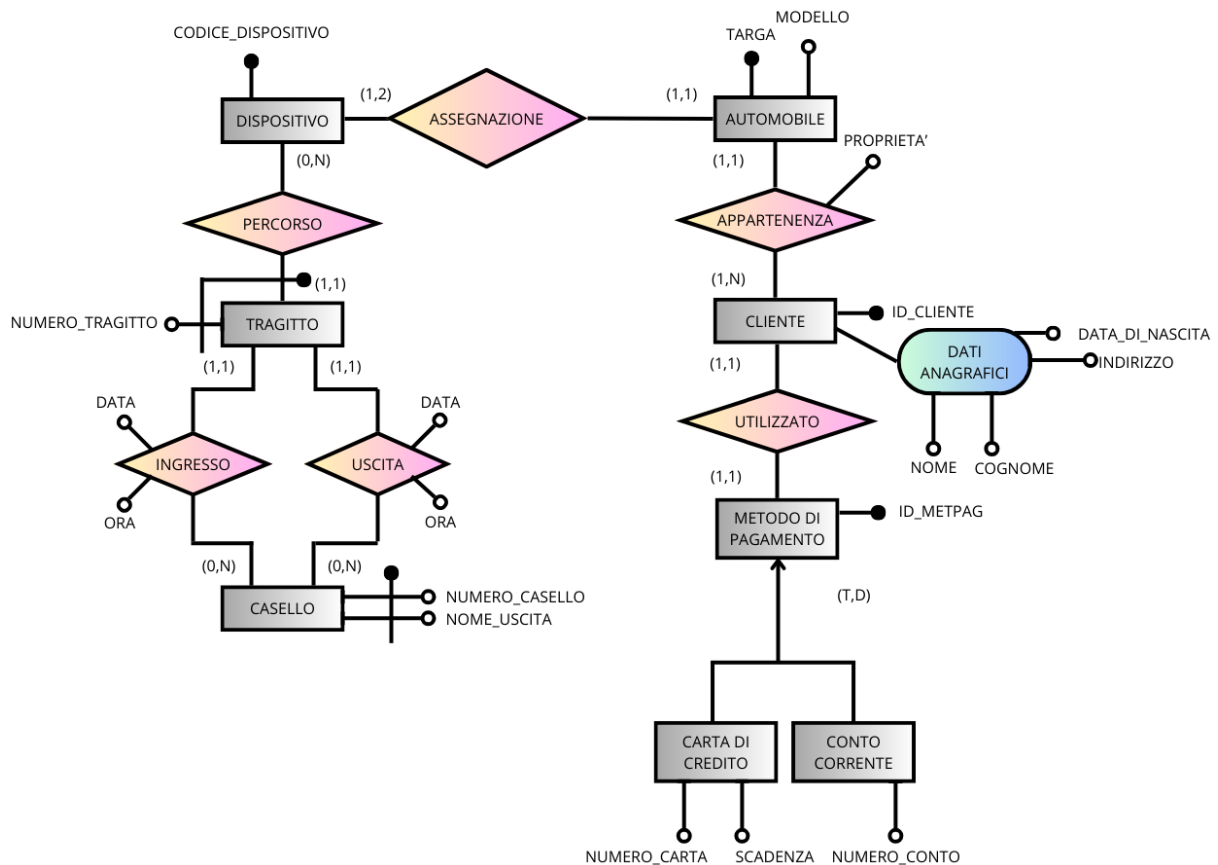
Additional relationships: INGRESSO (entry), USCITA (exit), APPARTENENZA (ownership), UTILIZZATO (used).

At this point all the attributes were added to the related entities and relationships, observing also that the METODO DI PAGAMENTO (payment method) can be specialised into CARTA DI CREDITO (credit card) and CONTO CORRENTE (current account), according to a total-disjoint hierarchy.

As for the cardinalities, it follows that:

- A DEVICE can be associated with a maximum of 2 cars (1,2), while each car can be associated with only one device (1,1), so the cardinality ratio is one-to-many.
- A CAR can be registered to only one customer (1,1), while a customer can own several cars (1,N), so in this case too we have a one-to-many cardinality.
- A CUSTOMER can use one payment method (1,1) and a payment method can be chosen by the customer (1,1), so the cardinality is one-to-one.
- A JOURNEY can be identified only by the device that travelled it (1,1), while a device can travel many journeys (0,N) (the 0 also accounts for devices that have not made any journey yet), so the cardinality is one-to-many.

- A TOLL BOOTH may also not be used for any journey (0,N), and every journey necessarily has one entry toll booth and one exit toll booth (1,1).



Complete ER schema. Labels are kept as they appear in the database schema (e.g. *PROPRIETA'* = ownership, *DATI ANAGRAFICI* = personal details, *DATA/ORA* = date/time, *SCADENZA* = expiry).

3.3 LOGICAL DESIGN

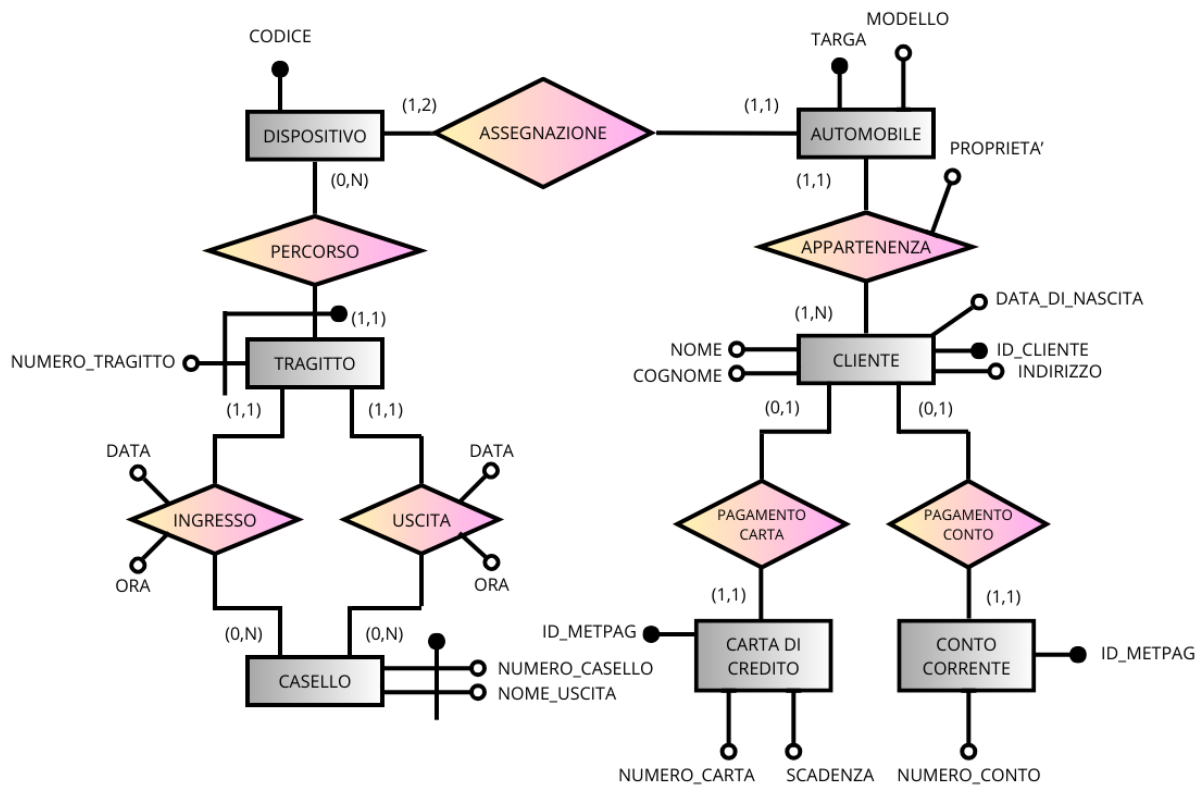
The conceptual design was followed by the logical design, aimed at translating the obtained ER schema into the corresponding relational model, i.e. a set of relations and constraints. This process consists of two phases:

3.3.1 TRANSFORMATION PHASE

In this phase the ER model is simplified in order to remove all those constructs that have no direct counterpart in the relational model, such as composite and multi-valued attributes, and the concept of generalisation/specialisation.

In our particular case it was therefore necessary to:

- remove the only composite attribute, *DATI ANAGRAFICI* (personal details), connecting its parts directly to the *CLIENTE* entity;
- resolve the total-disjoint generalisation of *METODO DI PAGAMENTO* by merging the parent entity into the children, transferring to the latter all the attributes it had (in this case only *ID_METPAG*). This resolution technique was chosen because the generalisation is total, and also because the operations to be performed will concern only the children.



Restructured ER schema: PAGAMENTO CARTA (card payment) and PAGAMENTO CONTO (account payment).

3.3.2 TRANSLATION PHASE

In the translation phase, the simplified schema obtained in the previous step is actually translated into the corresponding relational model.

In this phase every entity is translated into a relation whose attributes are the attributes of the entity and whose primary key is the identifying attribute of the entity.

In practice, the relationships ingresso, uscita, percorso, assegnazione, appartenenza, pagamento carta and pagamento conto were translated into inter-relational foreign key constraints. In addition, the attributes of the ingresso and uscita relationships, and the foreign key referencing the CASELLI relation, were added to the TRAGITTO entity. Finally, the related foreign keys and relationship attributes were also added to the AUTOMOBILI, CARTE DI CREDITO and CONTI CORRENTI relations (the attribute proprietà — ownership — in AUTOMOBILI).

The final relational schema is therefore:

DISPOSITIVI (Codice Dispositivo)

CASELLI (Numero Casello, Nome_Uscita)

CLIENTI (Id_Cliente, Nome, Cognome, Via, Città, Civico, CAP)

AUTOMOBILI (Targa, Modello, Cliente:CLIENTI, Proprieta, Codice_Dispositivo:DISPOSITIVI)

CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente:CLIENTI)

CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente:CLIENTI)

TRAGITTI (Numero_Tragitto, Codice_Dispositivo:DISPOSITIVI, Data_In,
Data_Out, Ora_In, Ora_Out, Numero_Casello_In:CASELLI,
Numero_Casello_Out:CASELLI)

The relational schema is not in 3NF because there are transitive dependencies (for example $CAP \rightarrow CITTA'$, i.e. postcode $\rightarrow$ city).

3.4 PHYSICAL DESIGN

```
CREATE TABLE DISPOSITIVI(  
  Codice_Dispositivo INTEGER,  
  CONSTRAINT PK_DISPOSITIVI PRIMARY KEY (Codice_Dispositivo)  
);  
  
CREATE TABLE CASELLI(  
  Numero_Casello INTEGER,  
  Nome_Uscita VARCHAR(30),  
  Id_Casello INTEGER,  
  CONSTRAINT PK_CASELLI PRIMARY KEY (Id_Casello)  
);  
  
CREATE TABLE CLIENTI(  
  Nome VARCHAR(20) NOT NULL,  
  Cognome VARCHAR(20) NOT NULL,  
  Id_Cliente INTEGER,  
  Via VARCHAR(15),  
  Citta VARCHAR(15),  
  Civico INTEGER,  
  CAP INTEGER,  
  CONSTRAINT PK_CLIENTI PRIMARY KEY (Id_Cliente)  
);  
  
CREATE TABLE AUTOMOBILI(  
  Targa VARCHAR(7),  
  Modello VARCHAR(30),  
  Cliente INTEGER NOT NULL,  
  Proprieta VARCHAR(2),  
  Codice_Dispositivo INTEGER NOT NULL,  
  CONSTRAINT PK_AUTOMOBILI PRIMARY KEY (Targa)  
);  
  
CREATE TABLE CONTI_CORRENTI(  
  Id_Conto INTEGER,  
  Numero_Conto VARCHAR(20) NOT NULL,  
  Id_Cliente INTEGER,  
  CONSTRAINT PK_C_C PRIMARY KEY (Id_Conto)  
);  
  
CREATE TABLE TRAGITTI(  
  Numero_Tragitto INTEGER,  
  Codice_Dispositivo INTEGER NOT NULL,  
  Data_In TIMESTAMP,  
  Data_Out TIMESTAMP,  
  Numero_Casello_In INTEGER,  
  Numero_Casello_Out INTEGER,  
  CONSTRAINT PK_TRAGITTI PRIMARY KEY (Numero_Tragitto, Codice_Dispositivo)  
);
```

Note that in the TRAGITTI relation the SQL TIMESTAMP type was used, which allows the Date and Time attributes to be merged, thus reducing the number of attributes.

We now add the referential integrity constraints:

```

ALTER TABLE AUTOMOBILI
ADD CONSTRAINT FK_AUTOMOBILI_DISPOSITIVI
FOREIGN KEY (Codice_Dispositivo) REFERENCES DISPOSITIVI (Codice_Dispositivo);

ALTER TABLE AUTOMOBILI
ADD CONSTRAINT FK_AUTOMOBILI_CLIENTI
FOREIGN KEY (Cliente) REFERENCES CLIENTI (Id_Cliente)
ON DELETE CASCADE;

ALTER TABLE CARTE_DI_CREDITO
ADD CONSTRAINT FK_CDC_CLIENTI
FOREIGN KEY (Id_Cliente) REFERENCES CLIENTI (Id_Cliente)
ON DELETE CASCADE;

ALTER TABLE CONTI_CORRENTI
ADD CONSTRAINT FK_CC_CLIENTI
FOREIGN KEY (Id_Cliente) REFERENCES CLIENTI (Id_Cliente)
ON DELETE CASCADE;

ALTER TABLE TRAGITTI
ADD CONSTRAINT FK_TRAGITTI_DISPOSITIVI
FOREIGN KEY (Codice_Dispositivo) REFERENCES DISPOSITIVI (Codice_Dispositivo)
ON DELETE CASCADE;

ALTER TABLE TRAGITTI
ADD CONSTRAINT FK_TRAGITTI_CASELLO_IN FOREIGN KEY (Numero_Casello_In)
REFERENCES CASELLI (Id_Casello)
ON DELETE SET NULL;

ALTER TABLE TRAGITTI
ADD CONSTRAINT FK_TRAGITTI_CASELLO_OUT FOREIGN KEY (Numero_Casello_Out)
REFERENCES CASELLI (Id_Casello)
ON DELETE SET NULL;

```

As for the reaction policies, note that:

- when a customer is deleted, the data relating to the cars and payment methods associated with them are also deleted in cascade;
- when a device is deleted, all the data relating to the corresponding journeys are deleted in cascade, while the operation is rejected (NO ACTION policy) if there are still cars associated with it;
- when a toll booth is deleted, the value NULL is set in the journeys associated with it, thus keeping the possibility of partially reconstructing a customer's movements through the remaining information (date, time, other toll booth).

We now proceed with populating the database:

```

-- Populate the DISPOSITIVI table
INSERT INTO DISPOSITIVI (Codice_Dispositivo) VALUES (1);
INSERT INTO DISPOSITIVI (Codice_Dispositivo) VALUES (2);
INSERT INTO DISPOSITIVI (Codice_Dispositivo) VALUES (3);
INSERT INTO DISPOSITIVI (Codice_Dispositivo) VALUES (4);
INSERT INTO DISPOSITIVI (Codice_Dispositivo) VALUES (5);

-- Populate the CASELLI table
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (1, 'NAPOLI EST', 1);
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (2, 'NAPOLI EST', 2);
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (3, 'ROMA SUD', 2);
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (4, 'CASERTA', 1);
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (5, 'FUORIGROTTA', 1);
INSERT INTO CASELLI (Id_Casello, Nome_Uscita, Numero_Casello) VALUES (6, 'MILANO SUD', 3);

-- Populate the CLIENTI table
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Mario', 'Rossi', 1, 'Via Roma', 'Napoli', 10, 80125);
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Giulia', 'Bianchi', 2, 'Via del Corso', 'Milano', 3, 20100);
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Giuseppe', 'Esposito', 3, 'Piazza Grande', 'Bologna', 5, 60292);
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Claudia', 'Prezioso', 4, 'Viale Paradiso', 'Roma', 30, 00133);
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Monica', 'Bellucci', 5, 'Corso Malta', 'Trento', 15, 38123);
INSERT INTO CLIENTI (Nome, Cognome, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Roberto', 'Vecchioni', 6, 'Via del Sole', 'Catania', 22, 95128);

-- Populate the AUTOMOBILI table
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AB123CD', 'Fiat 500', 1, 'si', 1);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('NA926PO', 'Toyota Corolla', 2, 'no', 2);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AZ321BC', 'Audi A3', 1, 'no', 1);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('RT567PL', 'Ferrari Purosangue', 4, 'si', 3);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('GF398DE', 'Volkswagen Golf 8', 5, 'si', 4);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('DS390TR', 'Lexus ES', 6, 'no', 5);

-- Populate the CARTE_DI_CREDITO table
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (1, '1234567812345678', TO_DATE('2023-12-31', 'YYYY-MM-DD'), 1);
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (2, '9876543298765432', TO_DATE('2024-06-30', 'YYYY-MM-DD'), 2);
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (3, '6739294185739137', TO_DATE('2026-06-12', 'YYYY-MM-DD'), 4);
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (4, '3719193846305828', TO_DATE('2022-10-11', 'YYYY-MM-DD'), 5);
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (5, '7310482876157238', TO_DATE('2025-02-19', 'YYYY-MM-DD'), 6);

```

```

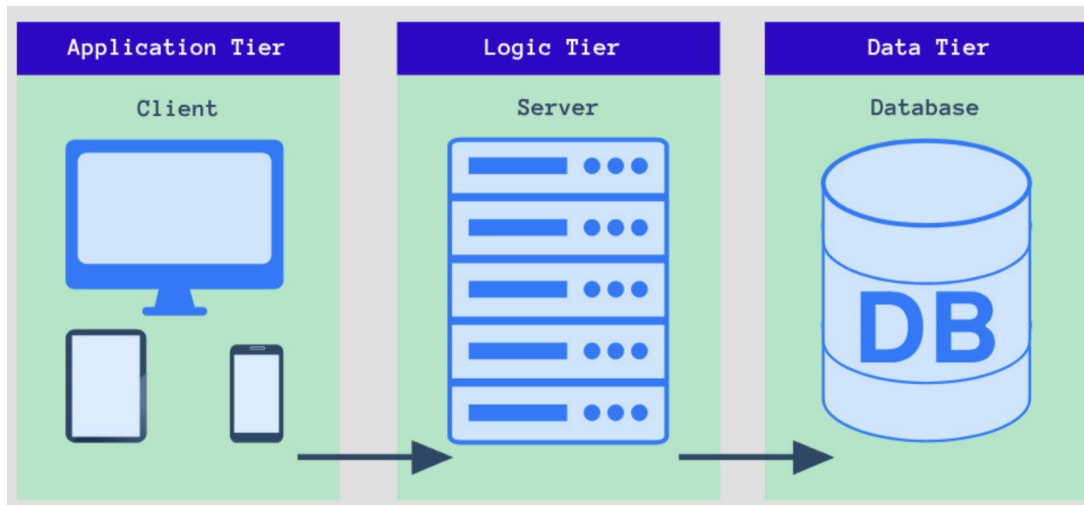
-- Populate the CONTI_CORRENTI table
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (1, '100100', 1);
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (2, '200200', 2);
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (3, '300300', 4);
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (4, '400400', 5);
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (5, '500500', 6);

-- Populate the TRAGITTI table
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (1, 1, '10-Sep-23 08-00-00', '10-Sep-23 09-00-00', 1, 2);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (2, 1, '3-May-22 10-00-00', '3-May-22 12-15-00', 2, 3);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (3, 2, '25-Dec-19 12-00-00', '25-Dec-19 12-30-00', 3, 1);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (4, 4, '30-Aug-23 11-30-00', '30-Aug-23 12-23-34', 4, 6);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (5, 1, '23-Jul-22 10-29-45', '24-Jul-22 01-05-23', 2, 3);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (6, 5, '31-Oct-23 07-20-22', '31-Oct-23 10-04-29', 6, 1);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Data_Out, Numero_Casello_In,
    Numero_Casello_Out) VALUES (7, 4, '01-Mar-24 06-45-32', '01-Mar-24 08-15-21', 2, 5);

```

4 APPLICATION DESIGN

The architecture planned for this database follows the “three-tier” model:



4.1 DATA TIER

This tier is responsible for managing data efficiently. It contains the DBMS, the related database and any other internal applications.

4.2 APPLICATION TIER

The application tier is responsible for transforming the user's input into the corresponding output. SQL code (interactive procedures) can also be embedded in the applications defined here, following one of these approaches:

- Embedded SQL;
- SQL/CLI;
- Database programming languages.

4.3 PRESENTATION TIER

This tier contains the interfaces that allow interaction with the user or with other application programs, and through which the results of the requested processing are returned.

5 OPTIMISATION

5.1 CHOOSING THE CONCURRENCY CONTROL METHOD

Concurrency management is a fundamental aspect to consider for the correct operation of a DBMS: without it, anomalies could occur that would leave the database in an inconsistent state. To avoid such scenarios, DBMSs implement concurrency control techniques that guarantee serialisable schedules (non-serial schedules, i.e. where operations of different transactions may overlap, but which

produce the same results as a serial schedule). To manage concurrency in our project we chose a lock-based method, specifically strict 2PL, according to which:

- every transaction must protect all reads and writes with locks;
- a transaction, after releasing a lock, cannot acquire any others;
- locks can be released only after commit or abort.

5.2 STRATEGIES TO ENSURE RELIABILITY

The reliability manager's goal is to restore the correct state (recovery) of a database following software or hardware failures due to human error, natural disasters or other causes. It must also guarantee the atomicity and durability properties of transactions, which underpin the recovery activities themselves, and this is where the main problem lies: the effects of a transaction committed in main memory are not immediately saved to secondary storage and made persistent — this is not an atomic operation — so if a failure occurs during this intermediate phase, vital information could be lost. To prevent this, reliability managers use a log file, i.e. a file saved in stable storage in which the actions performed by all transactions are recorded in their order of execution. Through the log it is possible to restore the state of a database after one of the scenarios mentioned above, by redoing the operations of those transactions that completed successfully but whose effects had not yet been made permanent before the problem occurred.

Finally, in this process the manager carries out two other fundamental activities: checkpoints (which record the transactions active at a given moment) and dumps (the classic backup operation).

6 SQL

6.1 QUERIES

SQL queries are obtained by combining a projection (vertical decomposition of the table) with a selection (horizontal decomposition of the table); some examples are shown below:

Query 1: select all customers whose first name starts with 'G':

```
SELECT C.Nome
FROM CLIENTI C
WHERE C.Nome LIKE 'G%';
```

NOME
Giulia
Giuseppe

Query 2: select the first name and surname of customers who have a Fiat 500:

```
SELECT C.Nome, C.Cognome
FROM CLIENTI C JOIN AUTOMOBILI A
ON C.Id_Cliente= A.Cliente
WHERE A.Modello='Fiat 500';
```

NOME	COGNOME
Mario	Rossi

In this query it was necessary to perform a JOIN between the CLIENTI and AUTOMOBILI tables.

Query 3: select the first name, surname and device of customers who have made at least 3 journeys:

```
SELECT C.Nome, C.Cognome, T.Codice_Dispositivo
FROM (CLIENTI C JOIN AUTOMOBILI A ON C.Id_Cliente = A.Cliente)
JOIN TRAGITTI T ON A.Codice_Dispositivo = T.Codice_Dispositivo
GROUP BY C.Nome, C.Cognome, T.Codice_Dispositivo
HAVING COUNT (T.Numero_Tragitto) >= 3
```

NOME	COGNOME	CODICE_DISPOSITIVO
Mario	Rossi	1

Query 4: select the licence plates of the cars that entered toll booth 2 on 3 May 2022 at 10:00:00:

```
SELECT A.Targa, A.Modello
FROM ((AUTOMOBILI A JOIN DISPOSITIVI D ON A.Codice_Dispositivo=D.Codice_Dispositivo)
JOIN TRAGITTI T ON D.Codice_Dispositivo=T.Codice_Dispositivo)
WHERE T.Numero_Casello_In='2' AND T.Data_In='3-May-22 10:00:00';
```

TARGA	MODELLO
AB123CD	Fiat 500
AZ321BC	Audi A3

Query 5: select the most used entry toll booth:

```
SELECT Numero_Casello_In, COUNT (*) AS Utilizzi
FROM TRAGITTI T
GROUP BY Numero_Casello_In
HAVING COUNT (*) >= ALL(
    SELECT COUNT(*)
    FROM TRAGITTI T1
    GROUP BY T1.Numero_Casello_In
);
```

NUMERO_CASELLO_IN	UTILIZZI
2	3

Query 6: select the entry toll booths with a number of passages above the average:

```
SELECT Numero_Casello_In, COUNT (*) AS Numero_Passaggi
FROM TRAGITTI T
GROUP BY Numero_Casello_In
HAVING COUNT (*) >= (
    SELECT AVG(Numero_Passaggi)
    FROM (SELECT COUNT(*) AS Numero_Passaggi
          FROM TRAGITTI T
          GROUP BY Numero_Casello_In)
);
```

NUMERO_CASELLO_IN	NUMERO_PASSAGGI
2	3

In this nested query, the innermost query returns a list of counts for each entry toll booth, the second innermost one computes the average over the list of values returned by the innermost query, and the outermost query selects only the toll booth and the number of passages of the booths whose count is above the previously computed average.

Moreover, the intermediate query has the innermost query in its FROM clause because it computes the average over the list of counted values.

Query 7: display, in descending order, the number of cars for each customer:

```
SELECT C.Nome, C.Cognome, COUNT(A.Targa) AS Numero_Automobili
FROM CLIENTI C JOIN AUTOMOBILI A ON C.Id_Cliente = A.Cliente
GROUP BY C.Nome, C.Cognome
ORDER BY Numero_Automobili DESC;
```

NOME	COGNOME	NUMERO_AUTOMOBILI
Mario	Rossi	2
Monica	Bellucci	1
Giulia	Bianchi	1
Claudia	Prezioso	1
Roberto	Vecchioni	1

Query 8: select the first name and surname of the owners of cars with no journeys in the last year:

```

SELECT C.Nome, C.Cognome, T.Data_in, T.Data_out
FROM ((CLIENTI C JOIN AUTOMOBILI A ON C.Id_Cliente=A.Cliente)
LEFT JOIN TRAGITTI T ON A.Codice_Dispositivo=T.Codice_Dispositivo)
WHERE C.Id_Cliente NOT IN(
SELECT C2.Id_Cliente
FROM ((CLIENTI C2 JOIN AUTOMOBILI A2 ON C2.Id_Cliente=A2.Cliente)
JOIN TRAGITTI T2 ON A2.Codice_Dispositivo=T2.Codice_Dispositivo)
WHERE (SYSDATE-365)<T2.Data_Out
) OR T.Data_Out=NULL;

```

NOME	COGNOME	DATA_IN	DATA_OUT
Giulia	Bianchi	25-DEC-19 12.00.00.000000 PM	25-DEC-19 12.30.00.000000 PM
Claudia	Prezioso	-	-

6.2 VIEWS

6.2.1 View showing the number of journeys made by each customer:

```

CREATE VIEW TRAGITTI_PER_CLIENTE
AS SELECT COUNT(T.Numero_Tragitto) AS NUM_TRAGITTI, C.Nome AS NOME,
C.Cognome AS COGNOME, C.Id_Cliente AS CODICE_CLIENTE
FROM (CLIENTI C LEFT JOIN AUTOMOBILI A ON C.Id_Cliente = A.Cliente)
LEFT JOIN TRAGITTI T ON T.Codice_Dispositivo = A.Codice_Dispositivo
GROUP BY C.Id_Cliente, C.Nome, C.Cognome;

```

Let us check that it works:

```
SELECT * FROM TRAGITTI_PER_CLIENTE;
```

NUM_TRAGITTI	NOME	COGNOME	CODICE_CLIENTE
1	Giulia	Bianchi	2
4	Mario	Rossi	1
0	Giuseppe	Esposito	3

Usage example: we want to find the first name and surname of the customer with the highest number of journeys travelled.

```

SELECT NOME, COGNOME, NUM_TRAGITTI
FROM TRAGITTI_PER_CLIENTE
WHERE NUM_TRAGITTI = (SELECT MAX(NUM_TRAGITTI) FROM TRAGITTI_Per_CLIENTE);

```

NOME	COGNOME	NUM_TRAGITTI
Mario	Rossi	4

6.2.2 View showing the payment methods associated with customers:

```

CREATE VIEW METODI_DI_PAGAMENTO
AS SELECT C.Id_Cliente AS CLIENTE, CDC.Numero_Carta AS NUMERO_CARTA,
CC.Numero_Conto AS NUMERO_CONTO
FROM (CARTE_DI_CREDITO CDC RIGHT JOIN CLIENTI C ON CDC.Id_Cliente = C.Id_Cliente)
LEFT JOIN CONTI_CORRENTI CC ON C.Id_Cliente = CC.Id_Cliente
ORDER BY C.Id_Cliente;

```

Let us check that it works:

```
SELECT * FROM METODI_DI_PAGAMENTO;
```

CLIENTE	NUMERO_CARTA	NUMERO_CONTO
---------	--------------	--------------

1	1234567812345678	100100
2	-	200200
3	9876543298765432	-

Usage example: we want to find the number of customers who use a card and the number of customers who use an account:

```
SELECT COUNT(NUMERO_CARTA) AS CARTE, COUNT(NUMERO_CONTO) AS CONTI
FROM METODI_DI_PAGAMENTO;
```

CARTE	CONTI
2	2

6.2.3 View showing all the journeys of customers who are also the owners of the car:

```
CREATE VIEW Tragitti_Proprietari_con_ogni_Automobile AS
SELECT C.Id_Cliente, C.Nome AS Nome_Proprietario, C.Cognome AS Cognome_Proprietario,
T.Numero_Tragitto AS Num_Tragitto, A.Targa AS Automobile, A.Proprieta AS Proprieta,
T.Data_In AS Data_Ingresso, T.Data_Out AS Data_Uscita
FROM ((CLIENTI C JOIN AUTOMOBILI A ON C.Id_Cliente=A.Cliente)
LEFT JOIN TRAGITTI T ON A.Codice_Dispositivo=T.Codice_Dispositivo)
WHERE A.Proprieta='si';
```

Let us check that it works:

```
SELECT * FROM Tragitti_Proprietari_con_ogni_Automobile;
```

ID_CLIENTE	NOME_PROPR.	COGNOME_PROPR.	NUM_TRAGITTO	AUTOMOBILE	PROPRIETA	DATA_INGRESSO	DATA_USCITA
1	Mario	Rossi	1	AB123CD	si	10-SEP-23 08.00.00 AM	10-SEP-23 08.00.00 AM
1	Mario	Rossi	2	AB123CD	si	03-MAY-22 10.00.00 AM	03-MAY-22 10.00.00 AM

Usage example: we want to find the licence plates of the cars of owner customers who have not made any journeys in the last year (but who have made at least one since they got the device):

```
SELECT T.Nome_Proprietario, T.Cognome_Proprietario, T.Automobile AS Auto_Senza_Tragitto
FROM Tragitti_Proprietari_con_ogni_Automobile T
WHERE T.Data_Uscita < SYSDATE - 365;
```

6.3 TRIGGERS

Triggers are special stored procedures kept inside the DBMS that are activated following an “event”. Their behaviour is based on the E-C-A paradigm, i.e. Event-Condition-Action. In particular, the type of triggering event allows triggers to be classified into DML triggers and DDL triggers. Below we show the definition and use of several DML triggers.

6.3.1 Entry/Exit date check:

```

CREATE OR REPLACE TRIGGER DATE_TRAGITTO_CONSISTENTI
BEFORE INSERT OR UPDATE ON TRAGITTI
FOR EACH ROW
DECLARE
    ERRORE EXCEPTION;
BEGIN
    IF :NEW.Data_Out <= :NEW.Data_In THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20001,
            'The exit date must be later than the entry date');
END;

```

Example:

```

INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo,
    Data_In, Data_Out, Numero_Casello_In, Numero_Casello_Out)
VALUES (10, 1, '20-Sep-23 08-00-00', '10-Sep-23 09-00-00', 1, 2);

```

```

ORA-20001: The exit date must be later than the entry date ORA-06512:

```

6.3.2 Check that each device has at most 2 cars belonging to a single customer.

```

CREATE OR REPLACE TRIGGER NUMERO_AUTO
BEFORE INSERT OR UPDATE ON AUTOMOBILI
FOR EACH ROW
DECLARE
    NUMERO_AUTO INTEGER;
    ALTRO_CLIENTE INTEGER;
    ERRORE_NUM_AUTO EXCEPTION;
    ERRORE_CLIENTI_DIVERSI EXCEPTION;
BEGIN
    --Check the number of cars already associated with the device
    SELECT COUNT(*) INTO NUMERO_AUTO FROM AUTOMOBILI
    WHERE Codice_Dispositivo = :NEW.Codice_Dispositivo; --AND CLIENTE = NEW:CLIENTE;

    IF NUMERO_AUTO >= 2 THEN
        RAISE ERRORE_NUM_AUTO;
    END IF;

    --Check that the device is used by a single customer
    SELECT COUNT(*) INTO ALTRO_CLIENTE
    FROM AUTOMOBILI
    WHERE Codice_Dispositivo = :NEW.Codice_Dispositivo AND Cliente <> :NEW.Cliente;

    IF ALTRO_CLIENTE > 0 THEN --(The device is already associated with another customer)
        RAISE ERRORE_CLIENTI_DIVERSI;
    END IF;

EXCEPTION
    WHEN ERRORE_NUM_AUTO THEN
        RAISE_APPLICATION_ERROR(-20002, '2 cars are already associated with the same device');
    WHEN ERRORE_CLIENTI_DIVERSI THEN
        RAISE_APPLICATION_ERROR(-20002, 'The device belongs to another customer');
END;

```

Example 1:

```

INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AB123CD', 'Fiat 500', 1, 'si', 1);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('NA926P0', 'Toyota Corolla', 1, 'no', 1);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AZ321BC', 'Audi A3', 1, 'no', 1);

```

ORA-20002: 2 cars are already associated with the same device ORA-06512: at

Example 2:

```

INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AB123CD', 'Fiat 500', 1, 'si', 1);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('NA926P0', 'Toyota Corolla', 2, 'no', 1);

```

ORA-20002: The device belongs to another customer ORA-06512: at

6.3.3 Credit card expiry check.

```

CREATE OR REPLACE TRIGGER SCADENZA CARTA
BEFORE INSERT OR UPDATE ON CARTE_DI_CREDITO
FOR EACH ROW
DECLARE
    ERRORE EXCEPTION;
BEGIN
    IF :NEW.Scadenza <= SYSDATE THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20003, 'The credit card has already expired');
END;

```

Example:

```

INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (1, '1234567812345678', TO_DATE('2022-12-31', 'YYYY-MM-DD'), 1);

```

ORA-20002: The credit card has already expired ORA-06512: at

6.3.4 Check that the customer is of legal age.

```

CREATE OR REPLACE TRIGGER CLIENTE_MAGGIORENNE
BEFORE INSERT OR UPDATE ON CLIENTI
FOR EACH ROW
DECLARE
    ETA NUMBER;
    ERRORE EXCEPTION;
BEGIN
    ETA := MONTHS_BETWEEN(SYSDATE, :NEW.Data_Nascita)/12;
    IF ETA < 18 THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20004, 'The customer must be of legal age');
END;

```

Example:

```
INSERT INTO CLIENTI (Nome, Cognome, Data_Nascita, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Mario', 'Rossi', '18-Sep-2010', 5, 'Via Roma', 'Napoli', 10, 80125);
```

```
ORA-20004: The customer must be of legal age ORA-06512:
```

6.3.5 Check that a payment method exists when a car is inserted.

```
CREATE OR REPLACE TRIGGER METODO_PAGAMENTO_ESISTENTE
BEFORE INSERT OR UPDATE ON AUTOMOBILI
FOR EACH ROW
DECLARE
    NUM_CONTI INTEGER;
    NUM_CARTE INTEGER;
    ERRORE EXCEPTION;
BEGIN
    SELECT COUNT (*) INTO NUM_CARTE
    FROM CARTE_DI_CREDITO
    WHERE Id_Cliente = :NEW.Cliente;

    SELECT COUNT (*) INTO NUM_CONTI
    FROM CONTI_CORRENTI
    WHERE Id_Cliente = :NEW.Cliente;

    IF (NUM_CARTE = 0 AND NUM_CONTI = 0) THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20005,
            'The customer associated with the car just inserted has no payment method!');
END;
```

Example:

```
INSERT INTO CLIENTI (Nome, Cognome, Data_Nascita, Id_Cliente, Via, Citta, Civico, CAP)
VALUES ('Mario', 'Rossi', '18-Sep-2001', 5, 'Via Roma', 'Napoli', 10, 80125);
INSERT INTO AUTOMOBILI (Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
VALUES ('AZ321CC', 'Audi A3', 10, 'no', 2);
```

```
ORA-20002: The customer associated with the car just inserted has no payment method!
ORA-06512: at "SQL_ZPZYCFQFBBTKGFXFSBUOQRTMD.METODO_PAGAMENTO_ESISTENTE", line 19
```

6.3.6 Check that the customer has at most one credit card.

```
CREATE OR REPLACE TRIGGER LIMITE_CARTE
BEFORE INSERT ON CARTE_DI_CREDITO
FOR EACH ROW
DECLARE
    NUMERO_CARTE INTEGER;
    ERRORE EXCEPTION;
BEGIN
    SELECT COUNT(CDC.Id_Carta) INTO NUMERO_CARTE
    FROM CARTE_DI_CREDITO CDC JOIN CLIENTI C ON C.Id_Cliente = CDC.Id_Cliente
    WHERE C.Id_Cliente = :NEW.Id_Cliente;

    IF NUMERO_CARTE > 0 THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20006, 'The customer already has a card registered');
END;
```

Example:

```

INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (10, '1234907812345678', TO_DATE('2025-12-31', 'YYYY-MM-DD'), 5);
INSERT INTO CARTE_DI_CREDITO (Id_Carta, Numero_Carta, Scadenza, Id_Cliente)
VALUES (15, '5467812345678', TO_DATE('2025-12-31', 'YYYY-MM-DD'), 5);

```

```
ORA-20003: The customer already has a card registered ORA-06512: at
```

6.3.7 Check that the customer has at most one current account.

```

CREATE OR REPLACE TRIGGER LIMITE_CONTI
BEFORE INSERT ON CONTI_CORRENTI
FOR EACH ROW
DECLARE
    NUMERO_CONTI INTEGER;
    ERRORE EXCEPTION;
BEGIN
    SELECT COUNT(CC.Id_Conto) INTO NUMERO_CONTI
    FROM CONTI_CORRENTI CC JOIN CLIENTI C ON C.Id_Cliente = CC.Id_Cliente
    WHERE C.Id_Cliente = :NEW.Id_Cliente;

    IF NUMERO_CONTI > 0 THEN
        RAISE ERRORE;
    END IF;
EXCEPTION
    WHEN ERRORE THEN
        RAISE_APPLICATION_ERROR(-20007, 'The customer already has an account registered');
END;

```

Example:

```

INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (10, '199100', 5);
INSERT INTO CONTI_CORRENTI (Id_Conto, Numero_Conto, Id_Cliente) VALUES (15, '108000', 5);

```

```
ORA-20003: The customer already has an account registered ORA-06512: at
```

6.4 STORED PROCEDURES

Stored procedures are programs (in our case written in PL/SQL) stored in the DBMS, which therefore interact quickly with the data.

6.4.1 Stored procedure that identifies the busiest motorway exit and advises against it (toll booth with the most unfinished journeys):

```

CREATE OR REPLACE PROCEDURE USCITA_PIU_TRAFFICATA(
    USCITA_TRAFFICATA OUT CASELLI.Nome_Uscita%TYPE
) IS
BEGIN
    SELECT NOME INTO USCITA_TRAFFICATA
    FROM TRAGITTI_NON_CONCLUSI
    WHERE NUM_TRAGITTI = (SELECT MAX(NUM_TRAGITTI) FROM TRAGITTI_NON_CONCLUSI);

    DBMS_OUTPUT.PUT_LINE('We advise NOT to take the exit ' || USCITA_TRAFFICATA ||
        ' as it is the busiest!');
END;

```

The following view was used:

```
CREATE VIEW TRAGITTI_NON_CONCLUSI
AS SELECT COUNT(T.Numero_Tragitto) AS NUM_TRAGITTI, T.Numero_Casello_In, C.Nome_Uscita AS NOME
FROM TRAGITTI T JOIN CASELLI C ON T.Numero_Casello_In = C.Id_Casello
WHERE T.Numero_Casello_Out IS NULL
GROUP BY T.Numero_Casello_In, C.Nome_Uscita;
```

We now perform some INSERTs to test how the procedure works.

```
--Example
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Numero_Casello_In)
VALUES (5, 1, '20-Feb-24 08-30-00', 4);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Numero_Casello_In)
VALUES (5, 2, '20-Feb-24 08-30-00', 5);
INSERT INTO TRAGITTI (Numero_Tragitto, Codice_Dispositivo, Data_In, Numero_Casello_In)
VALUES (6, 2, '20-Feb-24 08-35-00', 5);
SELECT * FROM TRAGITTI_NON_CONCLUSI;
```

NUM_TRAGITTI	NUMERO_CASELLO_IN	NOME
2	5	FUORIGROTTA
1	4	CASERTA

We now check how the procedure works:

```
DECLARE USCITA CASELLI.Nome_Uscita%TYPE;
BEGIN
    USCITA_PIU_TRAFFICATA(USCITA);
END;
```

```
Statement processed.
. We advise NOT to take the exit FUORIGROTTA as it is the busiest!
```

6.4.2 Stored procedure that takes a customer's identification code as input and returns the number of journeys they have made

```
CREATE OR REPLACE PROCEDURE TRAGITTI_TOTALI(
    Cliente_Input IN CLIENTI.Id_Cliente%TYPE,
    Num_Tragitti_Output OUT INTEGER
) IS
BEGIN
    SELECT NUM_TRAGITTI INTO Num_Tragitti_Output
    FROM TRAGITTI_PER_CLIENTE --View
    WHERE CODICE_CLIENTE = Cliente_Input;

    DBMS_OUTPUT.PUT_LINE('The customer with ID = ' || Cliente_Input || ' has made '
    || Num_Tragitti_Output || ' journeys.');
```

END;

Usage example:

```
DECLARE TRAGITTI INTEGER;
BEGIN
    TRAGITTI_TOTALI(1, TRAGITTI);
END;
```

```
Statement processed.
The customer with ID = 1 has made 4 journeys.
```

6.4.3 Stored procedure for inserting a car into the database

```

CREATE OR REPLACE PROCEDURE Inserimento_Automobile(
    p_Targa IN AUTOMOBILI.Targa%TYPE,
    p_Modello IN AUTOMOBILI.Modello%TYPE,
    p_Cliente IN AUTOMOBILI.Cliente%TYPE,
    p_Proprieta IN AUTOMOBILI.Proprieta%TYPE,
    p_CodiceDispositivo IN AUTOMOBILI.Codice_Dispositivo%TYPE
)
IS
BEGIN
    INSERT INTO AUTOMOBILI(Targa, Modello, Cliente, Proprieta, Codice_Dispositivo)
    VALUES (p_Targa, p_Modello, p_Cliente, p_Proprieta, p_CodiceDispositivo);
    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error while inserting the car');
        ROLLBACK;
        RAISE;
END Inserimento_Automobile;

```

Example:

```

DECLARE
    v_Targa          AUTOMOBILI.Targa%TYPE := 'CZ963ZZ';
    v_Modello        AUTOMOBILI.Modello%TYPE := 'Chevrolet Matiz';
    v_Cliente        AUTOMOBILI.Cliente%TYPE := 2;
    v_Proprieta      AUTOMOBILI.Proprieta%TYPE := 'si';
    v_CodiceDispositivo AUTOMOBILI.Codice_Dispositivo%TYPE := 5;

BEGIN
    Inserimento_Automobile(v_Targa, v_Modello, v_Cliente, v_Proprieta, v_CodiceDispositivo);

    DBMS_OUTPUT.PUT_LINE('Car inserted successfully');

END;

```